

Computer Science
Expose Master Thesis

Formal Verification Approaches for Real-Time Operating System Schedulers

Author:	Xaver Gruber
Matriculation Number:	42304184
Supervisor:	<Supervisor Name>
Date:	November 5, 2025

List of Contents

List of Abbreviations *3*

1 Introduction *4*

2 Problem Statement and Research Objectives *5*

3 Methodology *6*

4 Preliminary Outline *7*

Bibliography *10*

List of Abbreviations

FVM: Formal Verification Method

RTOS: Real-Time Operating System

1 Introduction

Real-time operating systems (RTOS) operate modern technology devices, which need immediate system responses. The systems operate in cars, airplanes, spacecraft, robots, and medical devices because they execute commands with exact precision at constant speed. The response time for brake signals, flight controller commands, and pacemaker signals must not exceed the designated millisecond threshold because any delay can result in equipment damage.

The operating system scheduler functions as its time planner component to decide which program elements can execute during specific times. The system follows programmed deadlines to deliver predictable responses to all system events (Khajeh et al., 2023).

The reliability of an RTOS depends on a fixed and predictable execution plan. A single mistake in task sequence can trigger process changes and resource battles, which eventually lead to system failure. Traditional tests and debugging tools cannot cover all possible situations because the number of system states is simply too large. Safety-critical systems use mathematical proof methods to confirm scheduler accuracy and system operation because system failures result in harm to people or equipment (Fusi et al., 2020).

Formal verification methods (FVM) serve as an effective solution to resolve these problems. The system depends on mathematical models with precise definitions to demonstrate system functionality through complete proof generation, which testing methods cannot achieve. The verification methods of Model Checking, Theorem Proving, and Timed Automata enable users to verify system behavior for blocking, result production, and timing constraint fulfillment. These methods study system operation through simulation instead of actual system execution, which results in safer analysis with proven results. The execution of this concept faces obstacles because it requires personnel who possess expertise in theoretical principles and system design elements (Xu & Ma, 2021).

The study aims to create FMSs which verify the correct operation of real-time scheduling systems. The thesis presents multiple methods, which explain their functionality and assess their suitable applications and their limitations. The investigation conducts a systematic

review of previous studies to show how FMSs improve the reliability and security of modern real-time operating systems. The framework establishes a reliable base, which enables researchers to create dependable and predictable systems.

2 Problem Statement and Research Objectives

Operating system scientists face real-time scheduler development and validation as their primary scientific obstacle. The scheduler system distributes CPU resources through priority management and protection of shared system resources. The timing behavior in flight control systems, medical devices, and autonomous vehicle systems needs to stay predictable because unpredictable behavior would create safety risks in these critical applications. The system can experience major breakdowns whenever any modification occurs to its scheduling or synchronization protocols. The rising complexity of systems creates an explosion of possible system states, which makes traditional testing methods unable to check all possible system configurations. Tests only verify specific situations, but they do not confirm that the entire system functions correctly (Kohútka & Mach, 2023).

FVMs serve to address this deficiency. The authors use mathematical logic to define systems and establish proof-based verification for system properties, including termination, deadlock prevention, and timing constraint satisfaction. Model Checking, Theorem Proving, and Timed Automata verify that a scheduler operates correctly under all possible system conditions. The proof methods provide better assurance than software testing methods. The process of creating formal models requires extended periods of time, while state spaces grow too large for analysis, and software systems cannot be fully modeled. Moreover, there is no unified framework to systematically compare existing verification methods (Zhao et al., 2021).

This thesis examines existing formal verification approaches for real-time schedulers and analyzes them theoretically. The evaluation determines which scheduling methods work best for various system configurations and scheduling systems and reveals operational constraints. The focus is not only theoretical; the thesis also investigates how these methods can be integrated into ongoing development processes. The study presents existing scholarly data to determine which methods show stability, scalability, and understanding.

3 Methodology

Methodologically, the thesis adopts a theoretical and analytical framework grounded in a structured and extensive literature review. Rather than employing empirical or experimental techniques, it relies on conceptual analysis to systematically examine and compare existing scientific approaches, models, and verification tools. This process results in a unified theoretical framework that clarifies the essential features, benefits, and limitations of each method. The analysis is based on recent and authoritative sources from the fields of real-time operating systems, formal methods, software verification, and system architecture.

The study uses specific databases for its data collection through IEEE Xplore, SpringerLink, ACM Digital Library, and ScienceDirect. The analysis uses textbooks, dissertations, and white papers as additional sources to enhance its theoretical base. The selection criteria consist of four elements, which are recent publication date, clear methodological design, number of citations, and direct application to real-time scheduling systems. The collected literature is categorized by methodological approach and application domain, for example, model-based, proof-oriented, or hybrid approaches, and analyzed systematically. The framework establishes a systematic approach, which allows researchers to conduct standardized comparative evaluations.

Once the literature has been collected, a detailed examination of the identified methods follows. The assessment of formal methods needs evaluation based on three fundamental factors, which include formalization levels, property verification capabilities for deadlock freedom, termination and determinism, system size compatibility, and real-world implementation feasibility. In addition, it is assessed whether theoretical models can be applied to real operating system architectures. The evaluation process demands both critical assessment and comparative analysis. The work aims to present personal methods while evaluating their scientific testing methods. The analysis focuses on three main problems, which stem from state-space growth, abstraction level selection, and computational resource requirements. The systematic evaluation shows methods to apply formal verification for various real-time applications.

The study requires a theoretical synthesis of collected data as its next methodological stage. The analytical results are organized and related to one another to provide a holistic view of the various approaches and their role within real-time system investigation. The analysis identifies commonalities and distinctions between methods, together with their possible combined effects. The study addresses open questions through studies about verification process automation, software development cycle integration, and formal model–machine learning technique combination. The synthesis achieves two goals by presenting current knowledge while establishing directions for upcoming studies.

The investigation uses an exploratory method, which upholds scientific standards. The framework depends on logical reasoning, established conceptual frameworks, and analytical methods. The study excludes experiments and self-built prototypes because it focuses on developing methods that align with theoretical frameworks. The outcome aims to establish the particular benefits of formal verification for real-time schedulers and their working constraints. Thus, the thesis not only advances theoretical understanding but also offers a structured framework to guide future empirical studies or practical applications.

4 Preliminary Outline

1. Introduction

1.1 Background and Motivation

1.2 Problem Statement

1.3 Goal of the Work

1.4 Structure of the Work

2. Theoretical and Conceptual Foundations

2.1 Real-Time OS Fundamentals

  2.1.1 Scheduling Basics

  2.1.2 Task Management

2.2 Determinism and Safety

2.2.1 Timing Predictability

2.2.2 Correctness Assurance

2.3 Formal Verification Basics

2.3.1 Mathematical Models

2.3.2 Logical Representations

3. Formal Verification Methods

3.1 Core Verification Approaches

3.1.1 Model Checking Tools

3.1.2 Theorem Proving Concepts

3.2 Time-Based Models

3.2.1 Timed Automata

3.2.2 Temporal Logic Methods

3.3 Combined and Hybrid Methods

3.3.1 Integrated Frameworks

3.3.2 Alternative Strategies

4. Comparison and Evaluation

4.1 Evaluation Criteria

4.1.1 Performance Metrics

4.1.2 Applicability Factors

4.2 Comparative Analysis

4.2.1 Method Strengths

4.2.2 Identified Weaknesses

4.3 Practical Aspects

4.3.1 Scalability Issues

4.3.2 Tool Support Overview

4.4 Summary of Insights

4.4.1 Key Comparative Findings

4.4.2 Implications for Research

4.4.3 Practical Takeaways

5. Challenges and Research Gaps

5.1 Limitations of Formal Modeling

5.2 State-Space Explosion

5.3 Integration into Industry

5.4 Technological Barriers

5.5 Future Research Directions

6. Conclusion and Outlook

6.1 Summary of Key Findings

6.2 Answers to Research Questions

6.3 Scientific Contribution

6.4 Practical Relevance

6.5 Future Outlook

x

Bibliography

Fusi, M., Mazzocchetti, F., Farres, A., Kosmidis, L., Canal, R., Cazorla, F. J., & Abella, J. (2020). On the Use of Probabilistic Worst-Case Execution Time Estimation for Parallel Applications in High Performance Systems. *Mathematics*, *8*(3), 314. https://doi.org/10.3390/math8030314

Khajeh, S. A., Saberikamarposhti, M., & Rahmani, A. M. (2023). Real-Time Scheduling in IoT Applications: A Systematic Review. *Sensors*, *23*(1), 232. https://doi.org/10.3390/s23010232

Kohútka, L., & Mach, J. (2023). A New FPGA-Based Task Scheduler for Real-Time Systems. *Electronics*, *12*(8), 1870. https://doi.org/10.3390/electronics12081870

Xu, W., & Ma, D. (2021). A Framework for Model and Verification of Safety-Critical Operating System Based on ARINC653. *Electronics*, *10*(16), 1934. https://doi.org/10.3390/electronics10161934

Zhao, H., Luo, H., & Wu, Y. (2021). A Data-Driven Scheme for Fault Detection of Discrete-Time Switched Systems. *Sensors*, *21*(12), 4138. https://doi.org/10.3390/s21124138